

Orders And Order_Details Management System

Orders Table

Attribute	Data Type	Size
Order_ID	NUMBER	5
Customer_ID	NUMBER	5
Order_Date	DATE	—
Total_Amount	NUMBER	10,2
Order_Status	VARCHAR2	20

Order_Details Table

Attribute	Data Type	Size
Order_Detail_ID	NUMBER	5
Order_ID	NUMBER	5
Product_ID	NUMBER	5
Quantity	NUMBER	5
Unit_Price	NUMBER	10,2
Subtotal	NUMBER	10,2

Create Table for Order :

Creates the Orders table to store customer order information.

```
SQL> CREATE TABLE Orders (  
2     Order_ID NUMBER(5) PRIMARY KEY,  
3     Customer_ID NUMBER(5),  
4     Order_Date DATE,  
5     Total_Amount NUMBER(10,2),  
6     Order_Status VARCHAR2(20)  
7 );
```

Table created.

2.INSERT for Order Table:

Inserts customer order details such as order ID, customer ID, date, amount, and status.

```
SQL> INSERT INTO Orders VALUES (1001, 101, TO_DATE('22-09-2026','DD-MM-YYYY'), 1500.00, 'Delivered');
1 row created.

SQL>
SQL> INSERT INTO Orders VALUES (1002, 102, TO_DATE('22-09-2026','DD-MM-YYYY'), 2499.00, 'Shipped');
1 row created.

SQL>
SQL> INSERT INTO Orders VALUES (1003, 103, TO_DATE('23-09-2026','DD-MM-YYYY'), 899.00, 'Processing');
1 row created.

SQL>
SQL> INSERT INTO Orders VALUES (1004, 104, TO_DATE('23-09-2026','DD-MM-YYYY'), 3200.00, 'Delivered');
1 row created.

SQL>
SQL> INSERT INTO Orders VALUES (1005, 105, TO_DATE('24-09-2026','DD-MM-YYYY'), 1250.00, 'Pending');
1 row created.

SQL>
SQL> INSERT INTO Orders VALUES (1006, 106, TO_DATE('24-09-2026','DD-MM-YYYY'), 2750.00, 'Shipped');
1 row created.
```

```
SQL> select * from Orders;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
1001	101	22-SEP-26	1500	Delivered
1002	102	22-SEP-26	2499	Shipped
1003	103	23-SEP-26	899	Processing
1004	104	23-SEP-26	3200	Delivered
1005	105	24-SEP-26	1250	Pending
1006	106	24-SEP-26	2750	Shipped
1007	107	25-SEP-26	999	Processing
1008	108	25-SEP-26	1850	Delivered

```
8 rows selected.
```

Create Table For Order_Details:

Creates the Order_Details table to store product-wise order information.

```
SQL> CREATE TABLE Order_Details (
2     Order_Detail_ID NUMBER(5) PRIMARY KEY,
3     Order_ID NUMBER(5),
4     Product_ID NUMBER(5),
5     Quantity NUMBER(5),
6     Unit_Price NUMBER(10,2),
7     Subtotal NUMBER(10,2),
8     FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
9 );
```

```
Table created.
```

Insert Values foe Order_Details:

Inserts product ID, quantity, unit price, and subtotal for each order.

```
SQL> INSERT INTO Order_Details VALUES (1, 1001, 201, 2, 500.00, 1000.00);
1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES (2, 1001, 202, 1, 500.00, 500.00);
1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES (3, 1002, 203, 1, 2499.00, 2499.00);
1 row created.

SQL>
SQL> INSERT INTO Order_Details VALUES (4, 1003, 204, 3, 299.00, 897.00);
1 row created.
```

```
SQL> SELECT * FROM Order_Details;
```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	UNIT_PRICE	SUBTOTAL
1	1001	201	2	500	1000
2	1001	202	1	500	500
3	1002	203	1	2499	2499
4	1003	204	3	299	897
5	1004	205	2	1200	2400
6	1005	206	1	1250	1250
7	1006	207	2	1375	2750
8	1007	208	1	999	999

```
8 rows selected.
```

Update Operation:

Modifies existing order information such as the order status.

```
SQL> UPDATE Orders
  2  SET Order_Status = 'Shipped'
  3  WHERE Order_ID = 1005;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.
```

5.JOIN OPERATION

The INNER JOIN operation is used to combine the Orders and Order_Details tables.

The tables are connected using the common Order_ID attribute.

```
SQL> SELECT O.Order_ID,
2      O.Customer_ID,
3      O.Order_Date,
4      D.Product_ID,
5      D.Quantity,
6      D.Unit_Price,
7      D.Subtotal,
8      O.Total_Amount,
9      O.Order_Status
10 FROM Orders O
11 INNER JOIN Order_Details D
12 ON O.Order_ID = D.Order_ID
13 ORDER BY O.Customer_ID, O.Order_Date;
```

ORDER_ID	CUSTOMER_ID	ORDER_DATE	PRODUCT_ID	QUANTITY	UNIT_PRICE	SUBTOTAL
1001	101	22-SEP-26	201	2	500	1000
1001	101	22-SEP-26	202	1	500	500
1002	102	22-SEP-26	203	1	2499	2499
1003	103	23-SEP-26	204	3	299	897
1004	104	23-SEP-26	205	2	1200	2400

6. CUSTOMER ORDER HISTORY REPORT

The customer order history report displays customer, order, and product details together.

The report is generated using INNER JOIN and arranged using Customer_ID and Order_Date.

```

SQL> SELECT O.Order_ID,
2      O.Customer_ID,
3      O.Order_Date,
4      D.Product_ID,
5      D.Quantity,
6      D.Unit_Price,
7      D.Subtotal,
8      O.Total_Amount,
9      O.Order_Status
10 FROM Orders O
11 INNER JOIN Order_Details D
12 ON O.Order_ID = D.Order_ID
13 ORDER BY O.Customer_ID, O.Order_Date;

```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	PRODUCT_ID	QUANTITY	UNIT_PRICE	SUBTOTAL

TOTAL_AMOUNT ORDER_STATUS						

1001	101	22-SEP-26	201	2	500	1000
1500	Delivered					
1001	101	22-SEP-26	202	1	500	500
1500	Delivered					
1002	102	22-SEP-26	203	1	2499	2499
2499	Shipped					

ORDER_ID	CUSTOMER_ID	ORDER_DAT	PRODUCT_ID	QUANTITY	UNIT_PRICE	SUBTOTAL

TOTAL_AMOUNT ORDER_STATUS						

1003	103	23-SEP-26	204	3	299	897
899	Processing					
1004	104	23-SEP-26	205	2	1200	2400
3200	Delivered					
1005	105	24-SEP-26	206	1	1250	1250
1250	Shipped					

7.CONCLUSION

The system successfully manages customer product orders and stores order-related information. The relationship between Orders and Order_Details helps maintain product-wise order details. The generated report provides a clear view of customer order history.